

Exploiting Structure for Real-Time Robot Motion Planning and Control

Gabriel Olin

July 29, 2026

School of Computer Science
Department of Robotics
Carnegie Mellon University
Pittsburgh, Pennsylvania 15213

Thesis Committee:

Howie Choset, *co-chair*
Maxim Likhachev, *co-chair*
Andrea Bajcsy
Itamar Mishani

*Thesis submitted in partial fulfillment of the
requirements for the degree of Master of Science in Robotics*

Dedicated to my friends and family

Abstract

Real-time robot motion in dynamic and uncertain environments requires algorithms that can make effective decisions under strict computational constraints. High-fidelity dynamics, large search spaces, uncertainty over future outcomes, and the pursuit of optimal motion each impose substantial computational costs. These challenges become especially pronounced in partially observable and rapidly changing environments, where a robot must continually update its predictions and react before a previously computed plan becomes obsolete. This thesis presents two model-based frameworks that exploit structure to bridge deliberative planning and reactive control, enabling efficient closed-loop execution without solving a full planning problem at every step.

First, we study projectile interception, where a robot must begin moving after only a few noisy measurements of a fast-moving object’s trajectory. Our framework couples uncertainty estimates from an RGB-D tracking system with a sparse kinodynamic graph of executable motion primitives. As the projectile estimate evolves, the system rapidly reevaluates candidate motions and executes partial trajectories that remain robust across a distribution of possible future outcomes. This approach maintains millisecond-scale replanning and improves interception performance.

The second framework learns the parameters of convex optimization-based safety filters formulated as control barrier function quadratic programs (CBF-QPs). Imitation learning transfers goal-directed behavior from a computationally expensive global planner into a local reactive controller while retaining explicit model-based safety constraints. This combination shifts computation from online global search to offline learning, enabling efficient closed-loop execution with reduced planning effort. We show that, under matched online computational budgets, the learned safety filter improves performance in planar navigation environments with moving obstacles and in constrained manipulator-planning tasks.

Together, these contributions show how structured representations can preserve reasoning about dynamics, uncertainty, and safety while enabling efficient real-time robot motion.

Acknowledgments

I would like to thank my advisors, Max Likhachev and Howie Choset, for giving me the chance to learn and grow as a researcher under them during my time at Carnegie Mellon. Coming into graduate school new to robotics, their patience and advice gave me an amazing foundation on which to explore the field while pursuing ambitious problems. These past two years have been the most formative of my academic career, and I would not have become the roboticist I am today without their support.

Furthermore, I would like to thank my peers in the Search-Based Planning Lab and the Biorobotics Lab for the great discussions, late nights pushing for deadlines together, and constant jokes which made work fun and inspiring.

Finally, I would like to thank my friends and family for supporting me during my graduate studies, who made moving to a new city easy and enjoyable and always gave me something to look forward to at the end of a long week. You were a pillar of my success here and I could not have done it without you.

Table of Contents

List of Figures	vii
List of Tables	ix
1 Introduction	1
2 Background and Related Work	2
2.1 Real-Time Kinodynamic Motion Planning	2
2.2 Planning and Estimation under Uncertainty	2
2.3 Optimization-Based Safety Filters	3
2.4 Learning Structured Controllers	4
3 Think Fast: Real-Time Kinodynamic Belief-Space Planning for Projectile Interception	5
3.1 Motivation	5
3.2 Problem Formulation	6
3.3 Computing Belief over Goal States	8

3.4	State-time Belief Space Planning	10
3.5	Experiments	16
3.6	Discussion	17
3.7	Conclusion	17
3.8	Acknowledgment	18
3.9	Perception and State Estimation	18
4	Learning Adaptive Control Barrier Functions for Safe, Real-Time Motion Planning	23
4.1	Motivation	23
4.2	Problem Formulation	24
4.3	Case Study 1: Nonholonomic Vehicle with Moving Obstacles	26
4.4	Case Study 2: Constrained, Reactive, Manipulator Planning	31
4.5	Limitations	35
4.6	Conclusion	36
5	Conclusion	37

Bibliography	38
------------------------	----

List of Figures

Figure 3.1	ABB arm with a shield attached to its end-effector blocking a thrown projectile	6
Figure 3.2	Task Space Goal Poses	7
Figure 3.3	Action Tree for 2D Double Integrator	14
Figure 3.4	Blocking success rate of the belief-space planner and the naive baseline over 50 projectile throws.	17
Figure 3.5	Flowchart showing Perception and State Estimation	19
Figure 3.6	The left and right figures show 3D locations of the projectile at different throws. Measurement noise varies between throws.	20
Figure 3.7	Measurement noise covariance matrix R and state covariance matrix P over time for the throw shown on the left in Fig. Figure 3.6 . The diagonal values of R vary due to inaccurate locations in the middle of the throw; the diagonal values of P decrease as expected when more information is given.	21
Figure 3.8	Unlike the Color Filter [16] , the retrained YOLOv8 can detect the projectile in the early stage, even before the ball was thrown.	22
Figure 4.1	To approximate a real-time navigation system, we run a global planner at 1 Hz, and the CBF-QP controller at 100Hz. The CBF-QP parameters are parametrized with a lightweight neural network which is trained to imitate an expensive expert planner which is queried at every rollout state in simulation.	27
Figure 4.2	Challenging obstacle configurations where untrained CBF-QP gets trapped in local minima, even with planner guidance.	30
Figure 4.3	Average Success Rate between trained and untrained methods.	31

Figure 4.4 Average Performance Comparison (Successful Runs).	31
--	----

List of Tables

Table 4.1	Average planning statistics for the expert and online planners per query.	30
-----------	---	----

Chapter 1

Introduction

This is where you would place a fun and thoughtful introduction. For example I could address why it is so important that I am working on this Thesis L^AT_EX Template? Well, it is probably so that future students that take on the challenge of writing their thesis in L^AT_EX can get through the formatting a lot quicker, in that they don't have to worry about it at all.

Here I will also add some tips: (1) Keep everything organized by 'Type'. Note that all the chapters are added in separate files in the 'chapter' directory/folder. This is such that you can stay better organized. Similarly all the figures are in the 'figure' directory, feel free to make sub folders for each chapter, and all the supporting files such as the references or the abbreviation and symbol lists are in their own folder.

(2) As you start working on this document try and keep the formatting to the end, because every time you change the text, figures or tables might move, and it is just easiest to leave that till the end. Lastly (3) this document and the accompanying files are to help beginners and experts, so if you are comfortable just go a head and get going. There is always the help guide to fall back on, and if you save a copy of the original files you can always go back and look what I presented to you.

There are some hidden chapters that are uncommented (with the % in front of them) with a lot of figure and table examples that can be use full for you if you are not sure for to scale or add them properly. Relax, and good luck.

Chapter 2

Background and Related Work

2.1 Real-Time Kinodynamic Motion Planning

2.2 Planning and Estimation under Uncertainty

2.2.1 Projectile Interception and Robotic Catching

Recent efforts on projectile interception with robot manipulators have explored both optimization-based planning and learning-based approaches. Bäuml et al. [6] present one of the earliest demonstrations of kinematically optimal ball catching with a 7-DOF arm and 12-DOF hand. Their approach leverages stereo vision and an extended Kalman filter (EKF) for trajectory prediction, while solving for catch time and configuration as a nonlinear constrained optimization problem. Real-time feasibility is achieved via parallel SQP, supporting multiple catching behaviors.

Sintov and Shapiro [23] propose the Time-Based RRT (TB-RRT), which augments RRT with temporal information to enable rendezvous planning under timing constraints. The algorithm is validated on interception tasks such as a 1-DOF bat striking a ball and a 3-DOF robotic arm catching a moving object, demonstrating precise time-synchronized planning.

Natarajan et al. [16] propose a preprocessing-based kinodynamic motion planning framework that leverages the INSAT planner and dome discretization to precompute smooth trajectories offline, enabling online retrieval of collision-free intercept motions in under a millisecond. While effective under strict real-time constraints, this approach relies on open-loop planning, which only uses one trajectory prediction to plan, and does not reason about uncertainty.

Kim et al. [10] present a programming-by-demonstration method that learns models of object dynamics and grasping configurations, using dynamical systems to generate rapidly adaptable catching motions. Their method achieves successful catching of diverse non-spherical objects but requires extensive demonstration data and motion capture infrastructure to train reliable models. Although these approaches consider dynamics and time constraints in the

planning problem, they do not explicitly reason about uncertainty.

2.2.2 Belief-Space Planning under Uncertainty

In certain planning under uncertainty problems, the belief evolution of objects in the robot’s environment is decoupled from action selection, which enables efficient planning approximations. In the autonomous driving space, the problem of online belief space planning has been applied to driving scenarios with uncertain obstacle motion or pedestrian intention. Aoude et al. [2] introduce RRT-Reach, a sampling-based threat assessment framework for driver assistance at intersections. Coupled with a Bayesian intention predictor, RRT-Reach evaluates escape trajectories in real time to recommend evasive maneuvers. Their Threat Assessment Module (TAM) is validated in both autonomous and human-driven vehicle experiments, showing effective detection and avoidance of intersection violations. Aoude et al. [3] also extend this work to handle dynamic obstacles with uncertain motion patterns, combining Chance-Constrained RRT (CC-RRT) with Regression-Reduced Gaussian Processes (RR-GP). This approach enforces per-step probabilistic safety guarantees under Gaussian uncertainty and outperforms naive and nominal planners in real-time simulations.

Bai et al. [4] apply POMDP planning to autonomous driving in crowded environments, explicitly reasoning about pedestrian intentions. Their intention-aware online planner achieves real-time performance (3 Hz) and demonstrates robustness in dynamic multi-agent scenarios.

2.3 Optimization-Based Safety Filters

2.3.1 Adaptive and Learned Control Barrier Functions

Recent work has sought to reduce the myopia, conservatism and tuning burden of control barrier function (CBF) safety filters by adapting the barrier parameters with various optimization and machine learning techniques. Parwana and Panagou [18] formulate a CBF/-CLF quadratic program as a differentiable policy and optimize its class- $\mathcal{K}$ parameters over a time horizon using recursive-feasibility-guided updates. Kim *et al.* [11] similarly adapt discrete-time input-constrained CBF parameters online using an uncertainty-aware probabilistic ensemble model. While these methods improve feasibility and performance, they still require optimization with respect to a manually defined objective function which can be difficult to achieve desired behavior from.

A second line of work incorporates differentiable optimization and learning directly into the CBF pipeline. Ma *et al.* [14] embed an CBF-QP as a differentiable layer and learn an environment-dependent parameterization of the class- $\mathcal{K}$ function for generalization to novel obstacle configurations. BarrierNet [24] similarly introduces a trainable differentiable

HOCBF-QP layer with environment-dependent parameters to reduce conservativeness while preserving safety guarantees. These approaches, while achieving promising results, primarily optimize the QP layer with respect to a small number of static obstacles, and might not be expressive enough for longer horizon planning with many moving obstacles, where the interaction between different constraints needs to be considered.

Other work has improved the objective used by the safety filter rather than learning the barrier parameters directly. Morton and Pavone [15] propose Operational Space Control Barrier Functions (OSCBFs), showing that task-consistent filtering can reduce unnecessary degradation of manipulator task performance, but still ultimately relies on hand tuned static QP formulations. Lin and Xu [12] propose a curvature-guided safety filter that replaces Euclidean projection with a Hessian-weighted metric derived from a learned action-value function, improving long-term value preservation. This method is promising because long horizon optimality is encouraged through a learned Q-function, but this can be hard to obtain, which motivates our approach to learn from a strong expert prior on long horizon behavior.

2.4 Learning Structured Controllers

2.4.1 Imitation Learning from Privileged Experts

2.4.2 Differentiable Optimization Layers

Chapter 3

Think Fast: Real-Time Kinodynamic Belief-Space Planning for Projectile Interception

3.1 Motivation

Intercepting fast-moving objects requires split-second decisions under incomplete information. While a robot must begin moving toward an intercept even before the object’s trajectory is fully observed, sensors provide noisy and partial estimates, producing a range of possible future states. This makes the interception problem not only a matter of planning dynamically feasible motions but also of reasoning under uncertainty in real time.

Prior work in robotic interception has shown how dynamics and timing can be incorporated into planners for interception [6, 23], but these methods often have a narrow success rate when dealing with noisy sensing, as they do not reason about uncertainty. In other motion planning problems with moving obstacles, such as autonomous driving, [2–4] planners explicitly reason about evolving beliefs of moving agents to plan robust collision-avoiding trajectories, but often these planners can only run at several Hz. For close range interception, where planning and execution need to be done within a fraction of a second, it becomes necessary to develop even more efficient algorithms.

In this work, we introduce a tree-based kinodynamic planner in state–time space, constructed from motion primitives rather than extend operators. This structure compactly encodes reachability to multiple goals, supports fast value updates as the target belief evolves, and enables seamless switching between intercept candidates. We combine this planner with an Innovation-based Adaptive Estimation Adaptive Kalman Filter (IAE-AKF) to maintain a belief over target trajectories. We evaluate the approach with hardware experiments on a 6-DOF ABB IRB-1600 robot arm and a ZED 2i stereo camera, demonstrating robust interception under uncertainty.

3.2 Problem Formulation

Consider a small projectile that is launched towards a vertical plane 0.8 meters from the base of fixed-based robot manipulator. The state of the projectile is represented as $\rho = (\rho_p, \rho_v)$, where $\rho_p \in \mathbb{R}^3$ denotes its position and $\rho_v \in \mathbb{R}^3$ its velocity. As noisy sensor readings describing the position of the projectile come in, the manipulator must move its end effector to block it before it passes the plane, which covers the region we want to protect.

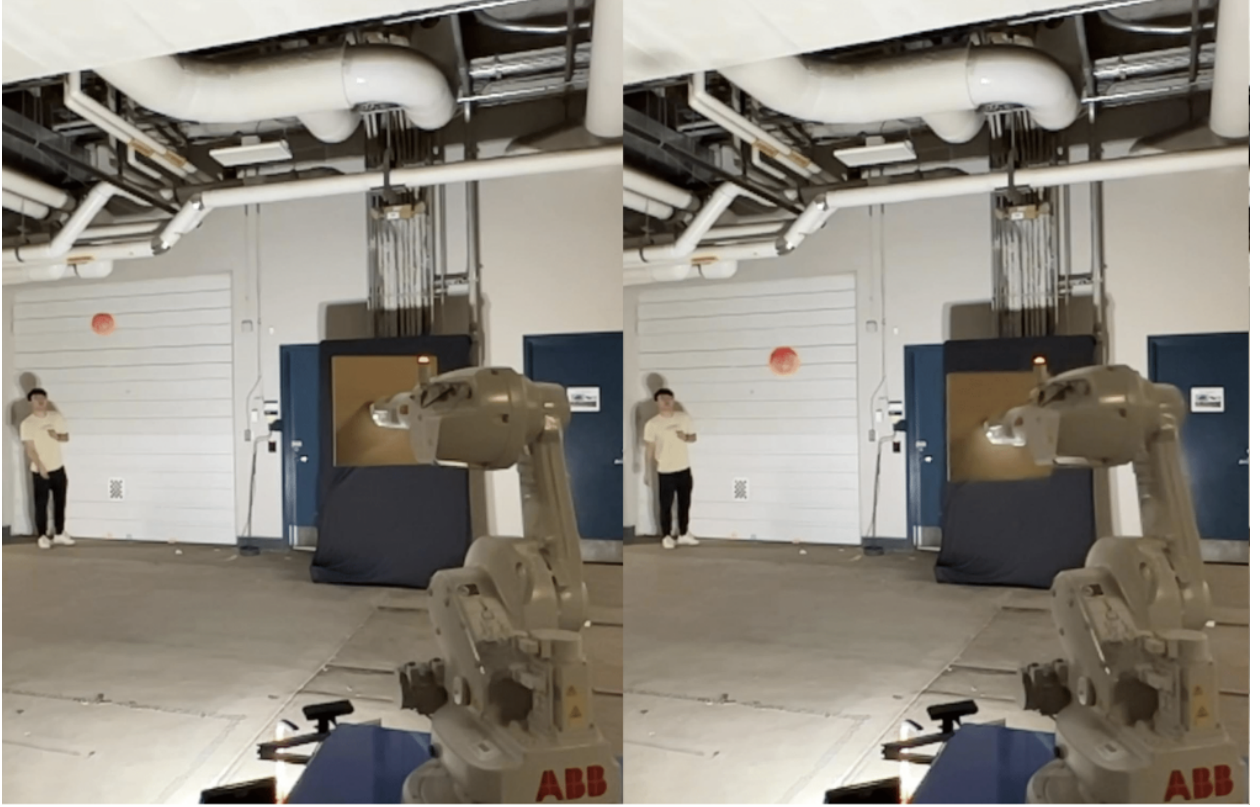


Figure 3.1. ABB arm with a shield attached to its end-effector blocking a thrown projectile

3.2.1 System Overview

Our manipulator is an ABB IRB-1600 robot arm with six DOF velocity control equipped with a rigidly attached shield to its end effector (Fig. [Figure 3.1](#)). To track and infer the state of the projectile, a ZED 2i stereo camera is attached to the base of the robot which gives regular RGB-D readings at 50Hz, which can be segmented into timestamped (x, y, z) coordinates. The intercept plane is chosen to be orthogonal to the camera's direction of view. To make the problem tractable in real time, we introduce the following simplifying assumptions:

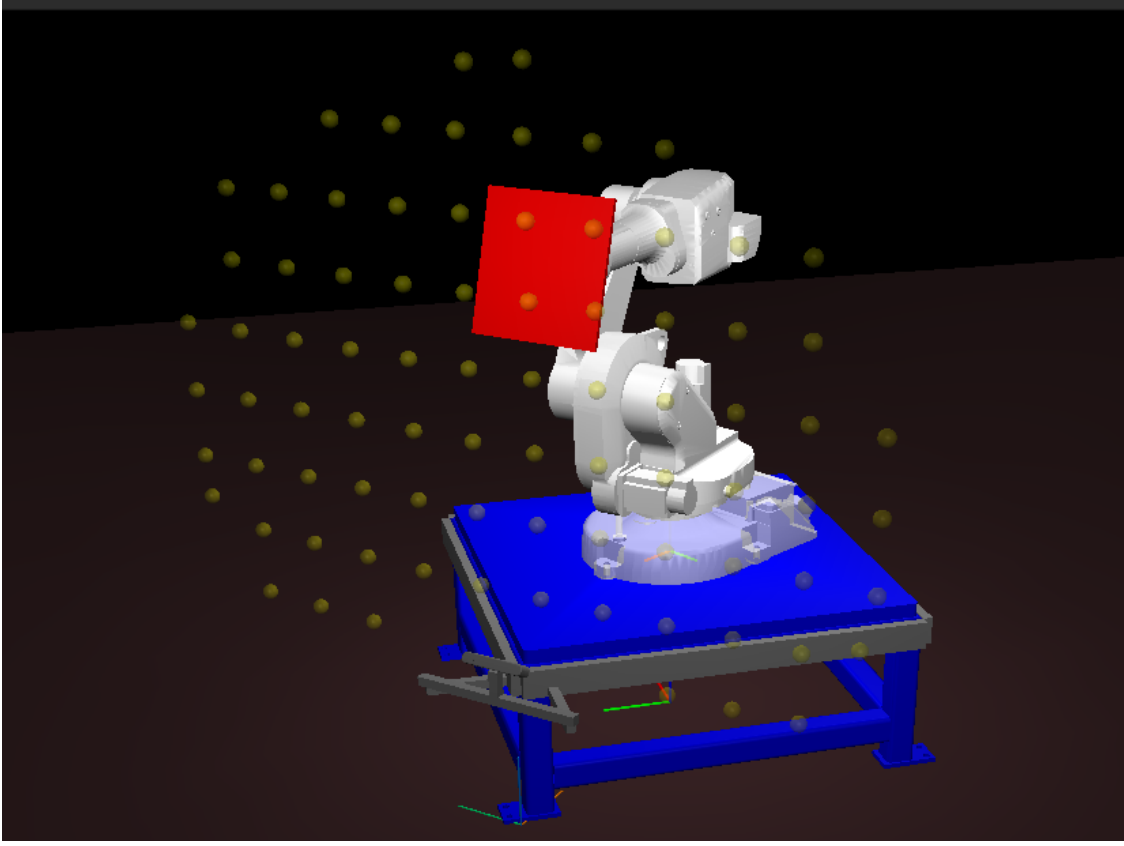


Figure 3.2. Task Space Goal Poses

- The manipulator always starts execution from a fixed “home” configuration q_{home} .
- Projectiles are launched from within the field of view of the onboard stereo camera, ensuring early state estimates of their trajectories.
- At any given time, only a single projectile is present in the environment.

3.2.2 Goal States

To cover the maximum amount of the robot’s workspace, we compute inverse kinematics solutions for the end effector for (y, z) coordinates at a 0.75 meter discretization in the plane frame (Fig. [Figure 3.2](#)). The orientation of the end effector is constrained so that the shield is coplanar with the plane. The joint configurations computed by a numerical IK solver are saved as goal states for the planner.

3.3 Computing Belief over Goal States

We consider a stochastic projectile state under gravitational acceleration g represented by a Gaussian belief, updated online by an Adaptive Kalman Filter (see Appendix)

$$s = \begin{bmatrix} X \\ Y \\ Z \\ V_x \\ V_y \\ V_z \end{bmatrix} \sim \mathcal{N}(\mu, \Sigma),$$

where (X, Y, Z) are positions and (V_x, V_y, V_z) are velocities, all expressed in the plane frame. We are interested in the time τ at which the trajectory crosses the interception plane $x = x^*$, as well as the corresponding lateral coordinates (Y, Z) at that crossing.

Although computing the probability distribution over the projectile's state and time when it crosses the intercept plane is generally computationally intractable due to its nonlinear dependence on V_x , the strict time budget of our problem necessitates a cheap analytical approximation to evaluate our belief over goal states online.

3.3.1 First-order Delta Method

The crossing time at the plane of interception in the robot's workspace is given by

$$\tau = \frac{x^* - X}{V_x}.$$

Using a first-order linearization about the mean state μ , the distribution of τ is approximated as Gaussian:

$$\mathbb{E}[\tau] \approx \bar{\tau} = \frac{x^* - \mu_X}{\mu_{V_x}}, \quad \text{Var}[\tau] \approx g_\tau \Sigma g_\tau^\top,$$

where the gradient of τ with respect to the state is

$$g_\tau = \begin{bmatrix} -\frac{1}{V_x} & 0 & 0 & -\frac{x^* - X}{V_x^2} & 0 & 0 \end{bmatrix}.$$

At the crossing time, the lateral coordinates are

$$Y_{\text{plane}} = Y + V_y \tau, \quad Z_{\text{plane}} = Z + V_z \tau + \frac{1}{2} g \tau^2,$$

where g is the gravitational acceleration (with the sign chosen by convention). Defining the output vector:

$$o = \begin{bmatrix} Y_{\text{plane}} \\ Z_{\text{plane}} \\ \tau \end{bmatrix},$$

we introduce the mapping $h : \mathbb{R}^6 \rightarrow \mathbb{R}^3$ such that

$$o = h(s),$$

Because $h : \mathbb{R}^6 \rightarrow \mathbb{R}^3$ is a nonlinear mapping between the current state and the crossing state/time, the distribution of $o = h(s)$ is non-Gaussian. The *delta method*[19] provides a first-order approximation by linearizing h about the mean μ :

$$h(s) \approx h(\mu) + J(s - \mu),$$

where

$$J = \left. \frac{\partial h}{\partial s} \right|_{s=\mu}$$

is the Jacobian of h evaluated at μ . The rows of $J \in \mathbb{R}^{3 \times 6}$ are given by the partial derivatives of $(Y_{\text{plane}}, Z_{\text{plane}}, \tau)$:

$$\begin{aligned} \frac{\partial Y_{\text{plane}}}{\partial s} &= \begin{bmatrix} -\frac{V_y}{V_x} & 1 & 0 & -\frac{V_y \tau}{V_x} & \tau & 0 \end{bmatrix}, \\ \frac{\partial Z_{\text{plane}}}{\partial s} &= \begin{bmatrix} -\frac{V_z}{V_x} - \frac{g\tau}{V_x} & 0 & 1 & -\frac{V_z \tau}{V_x} - \frac{g\tau^2}{V_x} & 0 & \tau \end{bmatrix}, \\ \frac{\partial \tau}{\partial s} &= \begin{bmatrix} -\frac{1}{V_x} & 0 & 0 & -\frac{x^* - X}{V_x^2} & 0 & 0 \end{bmatrix}. \end{aligned}$$

Under this linear approximation, o is approximately Gaussian:

$$o \sim \mathcal{N}(h(\mu), J\Sigma J^\top).$$

We apply this construction with $h(s)$ defined as the mapping from the projectile state $s = (X, Y, Z, V_x, V_y, V_z)$ to the lateral intercept coordinates and crossing time $o = (Y_{\text{plane}}, Z_{\text{plane}}, \tau)$.

3.3.2 Joint Gaussian at Plane

This construction yields a joint Gaussian approximation for the lateral coordinates at plane crossing and the crossing time:

$$(Y_{\text{plane}}, Z_{\text{plane}}, \tau)^\top \sim \mathcal{N}(\bar{o}, \Sigma_o),$$

which captures not only the individual variances but also the correlations between τ , Y_{plane} , and Z_{plane} induced by the uncertain initial state.

3.4 State-time Belief Space Planning

3.4.1 Action Tree

Offline, we construct an *action tree* rooted at the robot’s initial state in $\mathbb{R}^9$, where each node encodes a kinematic state consisting of joint position, velocity and acceleration (p, v, a) . Even though the ABB robot arm has six degrees of freedom, we plan only for the first three degrees of freedom because they account more most of the task space positional movement, which simplifies the planning problem. The orientation is controlled separately by moving the last three joints to align the shield with the incoming angle of the projectile. This decoupling is possible because the last three joints are able to move far faster and thus are not the limiting factor in the time-optimal control problem to move from one configuration to another.

From the start state, we iteratively expand the tree offline using minimum-time motion primitives, steering toward subgoals that span a discretization of the robot’s reachable front workspace plane. Primitives that are in collision are discarded and cut off at a maximum duration ϵ . Each expansion adds dynamically consistent primitives up to a fixed maximum depth, while storing time-to-come at each successor node. We choose the depth to limit the number of direction changes made while the projectile is in motion, since additional decision nodes can produce jerky motion, increase motor strain, and increase the cost of online evaluation. Nodes at the maximum depth are connected to nearby goal states with zero velocity.

The tree is not intended to densely cover the full robot state space. Instead, it covers the relevant state-time support of possible interception outcomes. Because a terminal outcome is described by an interception time and Cartesian intercept location, we voxelize the (T, Y, Z) domain. The (Y, Z) resolution is set by the end-effector shield size, while time is discretized over $[T_{\min}, T_{\max}]$. Times below $T_{\min}$ are omitted because no feasible trajectory can intercept the projectile that quickly; times above $T_{\max}$ are omitted because these outcomes are comparatively easy and less informative during tree construction.

Let

$$\Omega = \{(T_k, Y_k, Z_k)\}_{k=1}^K$$

denote this discretized state-time support, with voxel $\omega_k = (T_k, Y_k, Z_k)$. During preprocessing, we use a uniform prior over the bounded support, so $w_k = 1/K$. This preprocessing distribution is not assumed to be the true projectile distribution: at runtime, the estimated belief may be nonuniform and correlated.

For a tree node s reached at elapsed time $t(s)$, let $\tau_{\text{reach}}(s, Y_k, Z_k)$ be a fast estimate of the minimum remaining time required to reach (Y_k, Z_k) . Node s covers voxel ω_k when

$$t(s) + \tau_{\text{reach}}(s, Y_k, Z_k) \leq T_k.$$

Its coverage set is therefore

$$\mathcal{C}(s) = \{\omega_k \in \Omega : t(s) + \tau_{\text{reach}}(s, Y_k, Z_k) \leq T_k\},$$

and the coverage of a retained tree is

$$\mathcal{C}(\mathcal{T}) = \bigcup_{s \in \mathcal{T}} \mathcal{C}(s).$$

We seek a sparse tree that covers as much of this support as possible while limiting the number of retained nodes:

$$\mathcal{T}^* = \arg \max_{\mathcal{T}} \left[\sum_{\omega_k \in \mathcal{C}(\mathcal{T})} w_k - \lambda |\mathcal{T}| \right],$$

where λ penalizes the online computational cost of a larger tree. Under uniform voxel weighting, this objective becomes

$$\mathcal{T}^* = \arg \max_{\mathcal{T}} \left[\frac{|\mathcal{C}(\mathcal{T})|}{|\Omega|} - \lambda |\mathcal{T}| \right].$$

We do not solve this combinatorial optimization globally. Instead, Algorithms 1 and 2 apply a greedy approximation during steering. The high-level expansion proposes intermediate subgoals and generates minimum-time motion primitives, but retains a feasible child only if the newly covered probability mass exceeds the cost of another node.

The acceptance test

$$\Delta P(s') = \sum_{\omega_k \in \mathcal{C}(s') \setminus \mathcal{C}_{\text{covered}}} w_k > \lambda$$

retains only nodes that add meaningful, previously uncovered reachable mass. Thus, larger values of λ produce smaller trees with cheaper online evaluation, while smaller values retain more nodes and cover more possible interception outcomes. The resulting graph is a reusable, sparse motion-primitive tree over the support rather than a dense sample of robot state space. The action tree is acyclic by construction. As described in Section 3.5.3, online values are backed up from terminal nodes to the root in reverse topological order. Each retained node is processed once and each motion-primitive edge is evaluated once, requiring $O(N + M)$ work for N nodes and M edges.

Algorithm 1 Action Tree Construction

```
1: Input: Start state  $s_0$ , goal states  $G$ , goal regions  $\mathcal{R}$ , state limits  $\mathcal{X}$ , control limits  $\mathcal{U}$ ,  
   maximum primitive duration  $\epsilon$ , maximum depth  $d_{\max}$ , voxelized support  $\Omega$ , node penalty  
    $\lambda$   
2: Output: Action tree  $\mathcal{T}$   
3: Initialize tree  $\mathcal{T}$  with  $s_0$   
4: Initialize queue  $q \leftarrow \{s_0\}$   
5: Initialize covered set  $\mathcal{C}_{\text{covered}} \leftarrow \mathcal{C}(s_0)$   
6: while  $\mathcal{T}.\text{max\_level} < d_{\max}$  and  $q \neq \emptyset$  do  
7:    $s \leftarrow q.\text{pop}$   
8:   if  $s.\text{level} = d_{\max}$  then  
9:     CONNECTTOCLOSEGOALS( $s, G$ )  
10:    continue  
11:   end if  
12:   for all  $g \in \text{SUBGOALS}(s, \mathcal{R})$  do  
13:      $s' \leftarrow \text{STEERANDFILTER}(\mathcal{T}, s, g, \epsilon, \Omega, \mathcal{C}_{\text{covered}}, \lambda)$   
14:     if  $s' \neq \emptyset$  then  
15:        $q.\text{push}(s')$   
16:        $\mathcal{C}_{\text{covered}} \leftarrow \mathcal{C}_{\text{covered}} \cup \mathcal{C}(s')$   
17:     end if  
18:   end for  
19: end while  
20: return  $\mathcal{T}$ 
```

3.4.2 Minimum-time Optimal Control

Kinodynamic planning requires satisfaction of both kinematic constraints (e.g., position and velocity limits) and dynamic constraints (e.g., actuation, force, or torque bounds). However, for fully actuated robotic manipulators, exact dynamic feasibility is often unnecessary because low-level joint controllers can accurately track reference trajectories as long as they respect kinematic limits. For this reason, we approximate each joint's dynamics as a *triple integrator system*:

$$\dot{p} = v, \quad \dot{v} = a, \quad \dot{a} = u,$$

where u is the jerk control input.

The objective is to compute the minimum-time, jerk-limited trajectory connecting an initial state (p_0, v_0, a_0) to a terminal state (p_f, v_f, a_f) subject to state and control bounds. The problem can be formulated as:

Algorithm 2 STEERANDFILTER: Coverage-Regularized Steering

```
1: Input: Tree  $\mathcal{T}$ , parent state  $s$ , subgoal  $g$ , maximum duration  $\epsilon$ , voxelized support  $\Omega$ ,  
   covered set  $\mathcal{C}_{\text{covered}}$ , node penalty  $\lambda$   
2: Output: Accepted child node  $s'$  or  $\emptyset$   
3:  $\xi \leftarrow \text{MINIMUMTIMEPRIMITIVE}(s, g, \epsilon)$   
4: if  $\xi$  is infeasible then  
5:   return  $\emptyset$   
6: end if  
7:  $s' \leftarrow \xi.\text{final\_state}$   
8: Compute  $\mathcal{C}(s') \leftarrow \{\omega_k = (T_k, Y_k, Z_k) \in \Omega : t(s') + \tau_{\text{reach}}(s', Y_k, Z_k) \leq T_k\}$   
9:  $\Delta P(s') \leftarrow \sum_{\omega_k \in \mathcal{C}(s') \setminus \mathcal{C}_{\text{covered}}} w_k$   
    $\triangleright$  For uniform voxels,  $\Delta P(s') = |\mathcal{C}(s') \setminus \mathcal{C}_{\text{covered}}|/|\Omega|$ .  
10: if  $\Delta P(s') \leq \lambda$  then  
11:   return  $\emptyset$   
12: end if  
13: Add  $s'$  to  $\mathcal{T}$  as a child of  $s$   
14: Add primitive  $\xi$  as the edge from  $s$  to  $s'$   
15: return  $s'$ 
```

$$\begin{aligned} \min_{u(\cdot), T} \quad & J = T \\ \text{s.t.} \quad & \dot{a}(t) = u(t), \quad |u(t)| \leq u_{\max}, \\ & p_{\min} \leq p(t) \leq p_{\max}, \\ & v_{\min} \leq v(t) \leq v_{\max}, \\ & a_{\min} \leq a(t) \leq a_{\max}, \\ & p(0) = p_0, \quad v(0) = v_0, \quad a(0) = a_0, \\ & p(T) = p_f, \quad v(T) = v_f, \quad a(T) = a_f. \end{aligned}$$

From Pontryagin’s Minimum Principle, this formulation admits a bang–off–bang profile for the optimal control $u(t)$, which can be solved in closed form or via efficient numerical methods, providing minimum-time steering primitives for the action tree. We use the highly optimized Ruckig implementation [5] as our solver, because it can handle higher-order non-zero terminal constraints.

3.4.3 Value Function

To evaluate the quality of a candidate action, we define a *value function* over the state-time belief space. Each *terminal node* in the tree corresponds to a unique sequence of motion primitives that reaches a particular goal state $g \in \mathcal{G}$ with an associated arrival time τ_g .

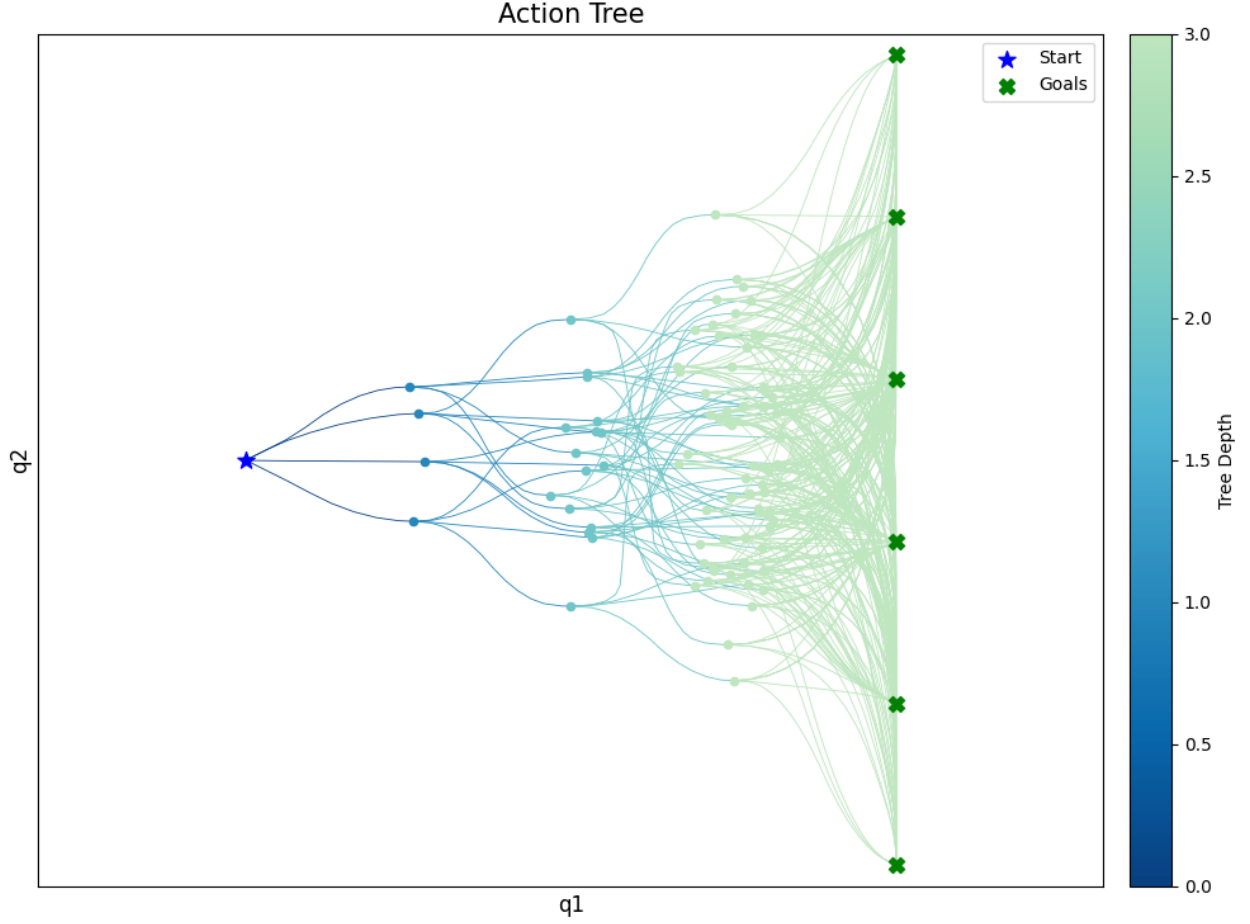


Figure 3.3. Action Tree for 2D Double Integrator

Let the belief over the lateral intercept coordinates and crossing time be represented by a Gaussian distribution

$$b(o) = \mathcal{N}(o \mid \bar{o}, \Sigma_o), \quad o = \begin{bmatrix} Y_{\text{plane}} \\ Z_{\text{plane}} \\ \tau \end{bmatrix}.$$

Because our camera is fixed to the base frame, our actions are not coupled to the observations we receive. This allows for the use of the QMDP approximation of the underlying POMDP, where our actions are the motion primitives in the tree, the beliefs are the Gaussian distribution over lateral intercept coordinates and time, and reward is a binary signal of whether the shield blocks the projectile before intercept time. Under the QMDP approximation, the value of a node n under belief $b(o)$ can be written as

$$V(n, b(o)) = \mathbb{E}_{o \sim b} \left[R(n, o) + \max_{a \in \mathcal{A}} V(n', b(o)) \right],$$

where $R(n, o)$ is the reward of being at node n

We define the reward function only for terminal nodes. For a terminal node n , let S denote the set of lateral coordinates (Y, Z) covered by the shield at the intercept plane. Then

$$R(n, o) = \begin{cases} 1, & (o_Y, o_Z) \in S \text{ and } n.\text{time} < o_\tau, \\ 0, & \text{otherwise,} \end{cases}$$

where $o = (o_Y, o_Z, o_\tau)$ are the intercept coordinates and the crossing time. This value function represents the expected probability of successfully intercepting the projectile, given the current distribution over intercept coordinates and crossing time.

The tree structure of our decision process allows terminal node values to be computed with each belief update and then propagated backward through the tree in a single online pass. Following topological value iteration [7], Algorithm 3 gives this procedure in a general form. In our action tree, the terminal reward $R_T(s)$ is the expected interception reward under the current belief, intermediate edge rewards are zero, and $\gamma = 1$. Thus, each belief update changes the terminal values, after which a reverse-topological Bellman backup recomputes the value of every decision node.

Each node is backed up once and each directed action edge is evaluated once. Therefore, for $N = |S|$ states and $M = |E|$ edges, the total work is

$$O(N + M).$$

Because G is acyclic, processing nodes in reverse topological order ensures that every successor value $V(s')$ is available before its parent s is backed up.

During execution, the next primitive is selected by choosing the successor of the current node that maximizes the value function:

$$a^* = \arg \max_{a \in \mathcal{A}(n)} V(n', b),$$

where $\mathcal{A}(n)$ denotes the set of available successors of node n , and n' is the child node reached by applying action a . If a terminal node is reached while projectile measurements are still being received, the planner tells the robot to move towards the most probable goal.

The closed-form Gaussian approximation of the belief state, along with efficient value updates in our action tree, allows for online planning. Our planner consistently computes the next action online in under 10ms on an AMD Threadripper Pro 5995WX workstation.

Algorithm 3 Reverse-Topological Bellman Backup

Require: Acyclic directed tree $G = (S, E)$
Require: Outgoing edge list $\text{Out}(s)$ for each $s \in S$
Require: Child node $\text{child}(e)$ and reward r_e for each edge $e \in E$
Require: Terminal reward $R_T(s)$ for each leaf s
Require: Discount factor γ
Require: Depth buckets $\mathcal{D}_0, \dots, \mathcal{D}_{d_{\max}}$
Ensure: Value function V

```
1: for all  $s \in S$  do
2:    $V(s) \leftarrow -\infty$ 
3: end for
4: for all  $s \in S_{\text{leaf}}$  do
5:    $V(s) \leftarrow R_T(s)$ 
6: end for
7: for  $d = d_{\max} - 1, d_{\max} - 2, \dots, 0$  do
8:   for all  $s \in \mathcal{D}_d$  do
9:     if  $\text{Out}(s) \neq \emptyset$  then
10:       $V(s) \leftarrow -\infty$ 
11:      for all  $e \in \text{Out}(s)$  do
12:         $s' \leftarrow \text{child}(e)$ 
13:         $Q_e \leftarrow r_e + \gamma V(s')$ 
14:         $V(s) \leftarrow \max\{V(s), Q_e\}$ 
15:      end for
16:    end if
17:  end for
18: end for
19: return  $V$ 
```

3.5 Experiments

We tested the full system with integrated planning, perception, and control on an IRB-1600 robot arm, equipped with a ZED 2i stereo camera in an indoor environment. Balls were thrown towards the robot’s front plane from distances from 6-8m. The time of flight of the throws were typically between 400ms and 800ms. We compared our method to a naive baseline, in which the robot moves (under kinematic constraints) as fast as possible to the current most likely goal configuration at each belief update.

Out of 50 throws of varying initial velocity and distance, our method achieved a blocking success rate of 74%. The naive approach still achieved 63% blocking success. The main cause of failure was insufficient execution time. While the perception module eventually returned an accurate intercept location, the end effector was too far away to reach it in time under its kinematic limits.

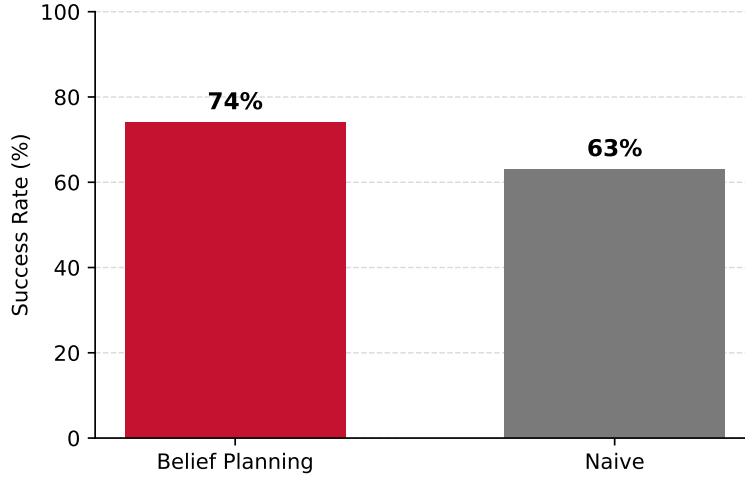


Figure 3.4. Blocking success rate of the belief-space planner and the naive baseline over 50 projectile throws.

3.6 Discussion

While both our method and the naive approach did not achieve 100% success rate, the improvement shows a benefit to considering uncertainty in extreme time-constrained interception problems. By encoding time-constrained reachability in the value function, our planner is able to hedge against a distribution of future observations, making it more robust than the naive approach which commits immediately to the first information it receives. Further work will include optimizing for manipulability of the robot arm in the motion primitives, which may allow for more agility when needing rapidly changing directions in task space. Additionally, the discrete action space may be limiting to when and how the robot can move, so optimization or learning based approaches may provide a strong continuous action space analog to our method.

3.7 Conclusion

We presented a real-time kinodynamic belief space planning framework for robotic projectile interception under uncertainty. By constructing an action tree in state-time space with jerk-limited motion primitives, our method enables fast online value updates as the projectile belief evolves, allowing the robot to seamlessly adapt its interception strategy. The QMDP-based value function, defined over the Gaussian distribution of intercept states, provides a principled approximation to reason about success probabilities under partial information.

Our integrated system—combining robust RIAE-AKF state estimation with kinodynamic motion primitives—demonstrated improved interception success over a naive baseline on hardware experiments with a 6-DOF ABB IRB-1600 manipulator. Despite inherent time constraints and occasional failures due to execution limits, the results highlight the importance of explicitly modeling uncertainty close range interception tasks.

Future work will explore incorporating robot manipulability into the value function, expanding from discrete primitives to continuous optimization or learning-based action spaces, and extending the framework to multi-interceptor scenarios. These directions could further enhance agility and robustness in interception tasks, bringing robotic systems closer to human-level performance in highly dynamic environments.

3.8 Acknowledgment

This work was supported by the Air Force Office of Scientific Research (AFOSR) under Grant M2403339 and, in part, by the Army Research Office (ARO) through the DURIP program under Grant W911NF-21-1-0050.

3.9 Perception and State Estimation

Perception is affected by multiple sources of noise, including low-resolution images, lighting variations, and complex backgrounds. Simple detection algorithms like a color filter, as used in [16], can struggle with robustness to these conditions. To reduce these effects and improve detection accuracy, we use a retrained YOLOv8 (You Only Look Once, version 8) [9] to detect the projectile in each frame using laboratory videos. The DINO grounding [13] was used as the base model, with AutoDistill CaptionOntology used to generate the training dataset. A custom annotation GUI was developed, allowing human verification and refinement of the ground truth. For each frame from the left ZED camera, the perception module detects the projectile and outputs its bounding boxes $[x_i, y_i, w_i, h_i]$, which contain the center point in the x and y axis and its width and height. To obtain the 3D location of the projectile, the detected bounding boxes are adjusted using a reverse sigmoid function based on the ratio of the bounding box area and image area. It keeps shrinking as the ball approaches the camera: 5.3% shrinkage in width and length when the ball is about 7-8 meters away, and up to 27.6% shrinkage when it is very close. These modified bounding boxes are then used to retrieve the projectile locations from ZED SDK’s confidence map and point clouds [16]. This adjustment reduces the noise from motion blur when the ball moves rapidly near the camera. The module outputs the projectile’s estimated 3D center location $[x_i, y_i, z_i]$.

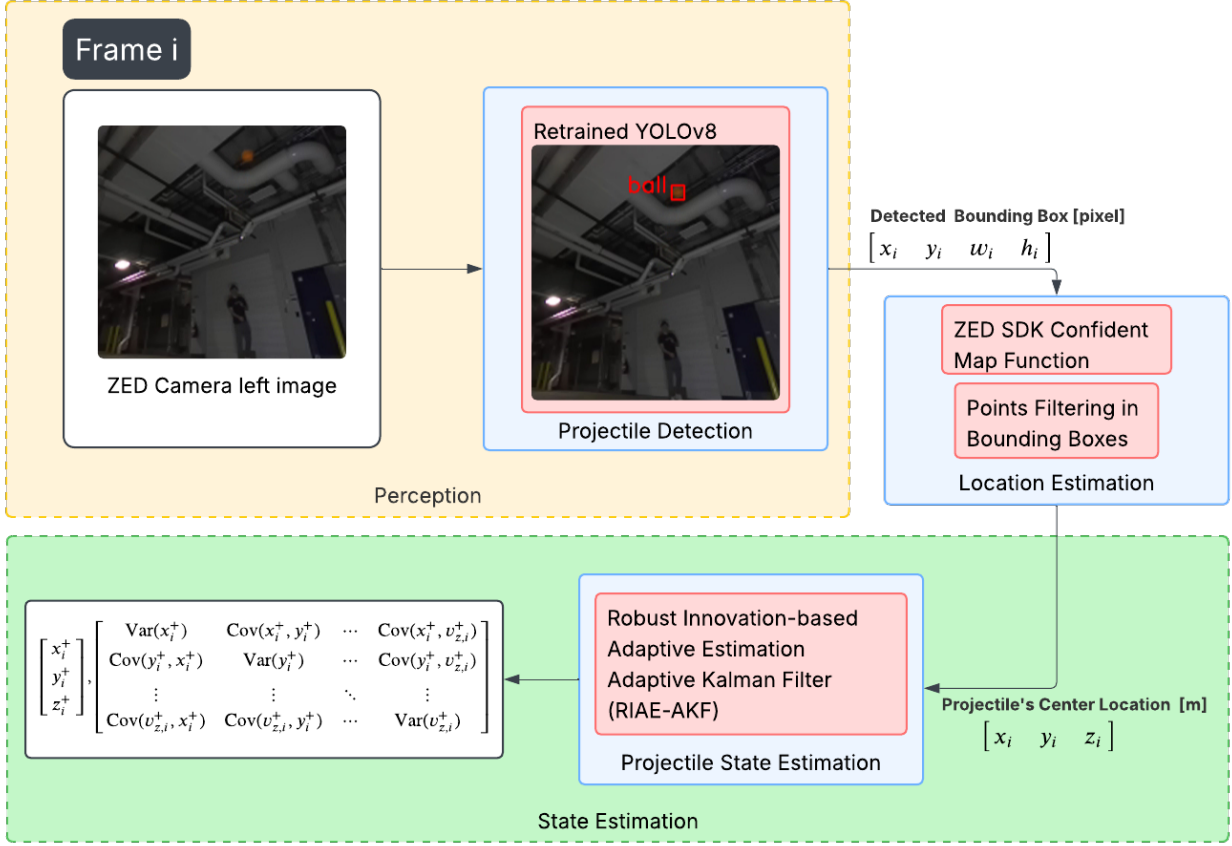


Figure 3.5. Flowchart showing Perception and State Estimation

A total of 1,408 images were used for training, including both high- and low-resolution cases, as well as scenarios in which the ball was held in the hand. The proposed perception module can detect the ball earlier than the color filter method (that is, before it is thrown), achieving an 81.3% detection success rate across 385 evaluation images, with an average center difference of 0.97 pixel units compared to the ground-truth bounding box center. In contrast, the color filter method achieved only a success rate of 38. 1% and a difference of 1.94 pixel units. An overview of the perception and state estimation pipeline is given in Fig. 2.

While detecting and segmentating the position of a projectile at several timesteps, a Kalman Filter (KF) can optimally estimate its state accounts for measurement and process noise with covariance matrices Q and R , but only when both are constant. In our case, different throws produce varying measurement noise (Fig. Figure 3.6), which makes the filtering process difficult. A constant process noise covariance is suitable since the model is only influenced by gravity ($g = 9.81 \text{ m/s}^2$). The innovation-based adaptive estimate adaptive Kalman filter (IAE-AKF) algorithm adjusts Q and R using the innovation history. However, when there

is disturbed noise or outliers, its residual can exceed Gaussian assumptions, leading to a degradation of performance [25]. To consider the varying measurement noise covariance and in the projectile's locations, we use Robust IAE-AKF (RIAE-AKF) [25] [22]

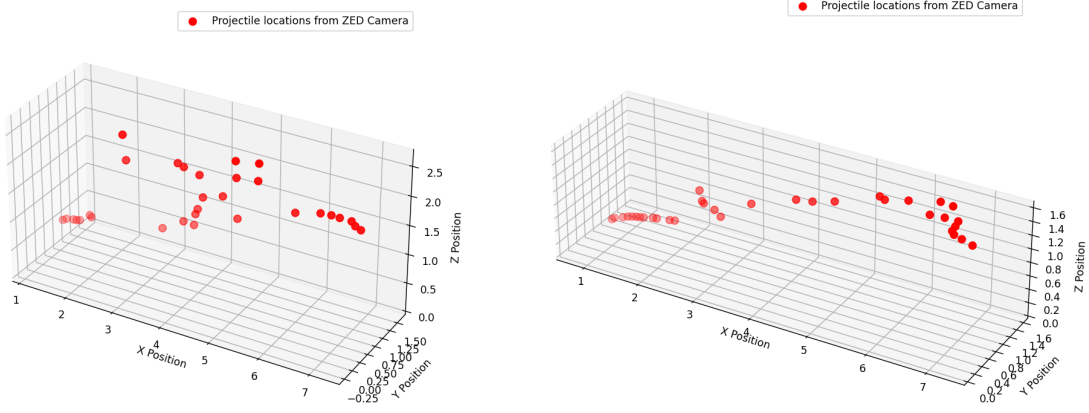


Figure 3.6. The left and right figures show 3D locations of the projectile at different throws. Measurement noise varies between throws.

Similar to the Kalman Filter, we need to calculate raw innovation $\boldsymbol{\nu}_k$ and covariance matrix after getting standard kalman prediction $\hat{\mathbf{x}}_k^-$ and $\mathbf{P}_k^-$.

$$\boldsymbol{\nu}_k = \mathbf{z}_k - \mathbf{H}_k \hat{\mathbf{x}}_k^- \quad (3.1)$$

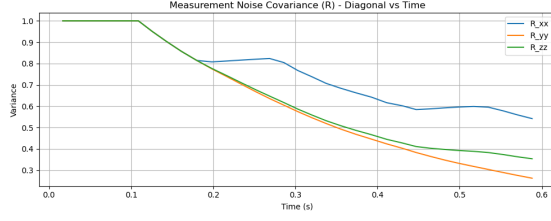
Where $\mathbf{H}_k$ is the observation matrix. For each element, calculate the chi-square statistic $\kappa_k(i)$ [8]:

$$\kappa_k(i) = \frac{\nu_k^2(i)}{\mathbf{C}_{v_{k-1}}(i, i)} \sim \chi^2(1) \quad (3.2)$$

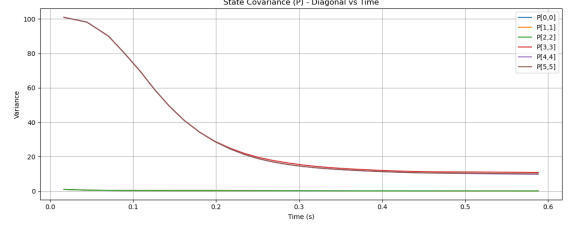
$\nu_k(i)$ is the i-th element of the vector ν_k , $\mathbf{C}_{v_{k-1}}(i, i)$ is the i-th diagonal element of empirical innovation covariance matrix $\mathbf{C}_{v_{k-1}}$, shown in Eq. (3.4). $\chi^2(m)$ is a Chi-square distribution with m degree of freedom, and m is the number of innovation vector's elements which we define as 1, assuming each dimension is independent to the other.

Next, we revise the innovation to reduce the influence of abnormal innovation vectors and update its covariance using the innovation history. Here, α represents the confidence level. The revised innovation is computed as

$$\hat{\nu}_k(i) = \begin{cases} \nu_k(i), & \text{if } 0 \leq \kappa_k(i) < \chi_\alpha^2(1) \\ \nu_k(i) \exp\left(-\frac{\kappa_k(i) - \chi_\alpha^2(1)}{\chi_\alpha^2(1)}\right), & \text{if } \kappa_k(i) \geq \chi_\alpha^2(1) \end{cases} \quad (3.3)$$



(a) Measurement Noise Covariance Matrix R



(b) State Covariance Matrix P

Figure 3.7. Measurement noise covariance matrix R and state covariance matrix P over time for the throw shown on the left in Fig. Figure 3.6. The diagonal values of R vary due to inaccurate locations in the middle of the throw; the diagonal values of P decrease as expected when more information is given.

When $0 \leq \kappa_k(i) < \chi_\alpha^2(1)$, the sensor reading is consistent with expectations. If $\kappa_k(i) \geq \chi_\alpha^2(1)$, the innovation is abnormal and will need to be modified using Eq. (3.3). Update Innovation covariance matrix $\mathbf{C}_{v,k}$:

$$\mathbf{C}_{v,k} = \frac{1}{N} \sum_{j=k-N+1}^k \hat{\mathbf{v}}_j \hat{\mathbf{v}}_j^\top \quad (3.4)$$

N is the window size and $\hat{\mathbf{v}}_j$ are the revised innovation vectors. Then we update the measurement noise covariance matrix $\mathbf{R}_k$.

$$\mathbf{R}_k = \mathbf{C}_{v,k} - \mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^\top \quad (3.5)$$

The State Estimation module adapts the measurement noise, recursively updates the state, and outputs the projectile state $\mathbf{x}_i^+ = [x_i^+, y_i^+, z_i^+, v_{x,i}^+, v_{y,i}^+, v_{z,i}^+]^T$ and the covariance matrix P :

$$P = \begin{bmatrix} \text{Var}(x_i^+) & \text{Cov}(x_i^+, y_i^+) & \cdots & \text{Cov}(x_i^+, v_{z,i}^+) \\ \text{Cov}(y_i^+, x_i^+) & \text{Var}(y_i^+) & \cdots & \text{Cov}(y_i^+, v_{z,i}^+) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(v_{z,i}^+, x_i^+) & \text{Cov}(v_{z,i}^+, y_i^+) & \cdots & \text{Var}(v_{z,i}^+) \end{bmatrix}.$$

To evaluate performance compared to the work [16], given the absence of ground-truth 3D projectile locations, we used the estimated depth results to compute the intersection with a virtual vertical wall placed 2 meters from the camera. This intersection point serves as a reference point for ground truth, since the measurements are empirically closer to the real trajectories of the projectile. The intersection predicted by RIAE-AKF is then calculated, and its distance from the reference point of the ground truth is calculated to compare with



(a) An example of a zoomed in frame received by the left ZED camera.



(b) Same frame with the projectile detected, which the Color Filter fails.

Figure 3.8. Unlike the Color Filter [16], the retrained YOLOv8 can detect the projectile in the early stage, even before the ball was thrown.

the work [16]. shows that the work [16] has fluctuated prediction compared to the proposed work.

Chapter 4

Learning Adaptive Control Barrier Functions for Safe, Real-Time Motion Planning

4.1 Motivation

Computing kinodynamically feasible trajectories for nonlinear robotic systems, and repairing them online, remains a central challenge in robot motion planning. This is especially important in cluttered and dynamic environments, where robots must replan quickly while respecting system dynamics, actuator limits, and collision-avoidance constraints. Although global or kinodynamic planners can produce high-quality trajectories, they are often too computationally expensive to run at high receding-horizon frequencies and may rely on simplified planning representations in real-time autonomy stacks.

A common strategy is to use an expensive planner as an expert and distill its behavior into a cheaper policy [21]. However, learned or simplified policies alone may not reliably preserve safety when deployed closed loop, particularly under distribution shift, moving obstacles, or nonlinear dynamics. Rather than learning an unconstrained end-to-end controller from demonstrations, our approach uses a Control Barrier Function Quadratic Program (CBF-QP) as a structured policy class that embeds known model information and safety constraints directly into the learning problem. Given a nominal control from a policy or planner, the CBF-QP minimally modifies the control to satisfy safety constraints under the system dynamics, providing an inductive bias toward dynamically feasible and safety-aware behavior. This reduces the burden on the neural network to infer such structure purely from data, which can be data inefficient and sensitive to architecture, optimization choices, and dataset coverage, particularly in safety-critical settings where rare but consequential constraint-violating states may be underrepresented.

The effectiveness of this hybrid approach depends critically on the parameters of the CBF-QP. Class- $\mathcal{K}$ gains affect how aggressively the controller responds to safety constraints, while quadratic objective weights determine which control directions are modified to maintain safety. These parameters are typically fixed and manually tuned, but fixed choices are often insufficient in cluttered, dynamic environments where good behavior depends on local geometry, obstacle motion, and task intent. Poorly tuned CBF-QPs can be overly conservative, locally myopic, or misaligned with the long-horizon behavior of the expert planner.

Our method wraps a computationally cheap planner with a learned CBF-QP safety filter, where a lightweight neural network predicts state-dependent QP parameters and the differentiable QP produces the executed control. In this way, the system distills expert behavior while preserving the structure of a model-based safety filter, enabling fast reactive control that better balances collision avoidance, dynamic feasibility, and goal progress.

4.2 Problem Formulation

We consider real-time kinodynamic motion planning for a nonlinear robotic system operating in a dynamic environment. Let $x \in \mathcal{X}$ denote the system state and $u \in \mathcal{U}$ denote the control input. For a control-affine system

$$\dot{x} = f(x) + g(x)u, \quad (4.1)$$

let $h_i(x)$ denote a control barrier function associated with safety constraint i , where the corresponding safe set is

$$\mathcal{C}_i = \{x \in \mathcal{X} : h_i(x) \geq 0\}.$$

A continuously differentiable function h_i is a control barrier function if there exists a class- $\mathcal{K}$ function α_i such that, for all x in the relevant domain, there exists a control $u \in \mathcal{U}$ satisfying

$$L_f h_i(x) + L_g h_i(x)u \geq -\alpha_i(h_i(x)). \quad (4.2)$$

A class- $\mathcal{K}$ function is a continuous, strictly monotonically increasing function $\alpha : [0, a) \rightarrow [0, \infty)$ satisfying $\alpha(0) = 0$. In the CBF constraint, α_i determines how aggressively the controller responds near the boundary of the safe set: larger values allow the system to approach the boundary more quickly, while smaller values enforce more conservative behavior near obstacles.

At each control step, the robot must make progress toward a goal while avoiding time-varying obstacles and respecting the system dynamics and control limits. An ideal kinodynamic planner solves a finite-horizon constrained optimization problem of the form

$$\begin{aligned}
\min_{\{u_k\}_{k=0}^{N-1}} \quad & J_D = \sum_{k=0}^{N-1} L(x_k, u_k, k) \\
\text{s.t.} \quad & x_{k+1} = f_D(x_k, u_k), \quad k = 0, \dots, N-1, \\
& x_0 = x_I, \\
& x_N \approx x_G, \\
& x_k \in \mathcal{X}_{\text{free},k}, \quad k = 0, \dots, N, \\
& u_k \in \mathcal{U}, \quad k = 0, \dots, N-1.
\end{aligned} \tag{4.3}$$

Here, f_D denotes the discrete-time system dynamics and $\mathcal{X}_{\text{free},k}$ is the collision-free subset of the state space at time step k , induced by the workspace obstacles. Solving (4.3) with expensive kinodynamic planners or optimal control solvers can produce high-quality behavior, but is often not feasible to run at high receding-horizon frequencies, or requires additional information only available in simulation. We therefore use these solvers only as a privileged expert to provide supervision for desired controls:

$$u_E = \pi_E(z),$$

where z denotes the local planning context, including the robot state, goal information, and nearby obstacle states.

At runtime, we assume access to a nominal task-directed control

$$u_{\text{ref}} \in \mathcal{U},$$

produced by a higher-level planning module. This module may output open-loop controls directly, or it may output a path or coarse trajectory that is tracked by a separate controller. We abstract these details and treat u_{ref} as the nominal control command available to the local controller.

Our goal is to learn a local safety-filtered policy

$$u_{\text{safe}} = \pi_{\text{safe},\theta}(z, u_{\text{ref}}),$$

that approximates the expert behavior while retaining the structure of a model-based CBF safety filter, allowing the robot to act intelligently between online planning queries.

Given expert demonstrations

$$\mathcal{D} = \{(z_j, u_{\text{ref},j}, u_{E,j})\}_{j=1}^{N_D},$$

we seek parameters θ such that

$$\pi_{\text{safe},\theta}(z_j, u_{\text{ref},j}) \approx u_{E,j}.$$

This formulation defines the proposed hybrid model-learning problem: the higher-level planner provides a cheap task-directed nominal command u_{ref} , the privileged expert provides the desired local behavior u_E , and the learned safety filter maps (z, u_{ref}) to the executed control u_{safe} .

4.3 Case Study 1: Nonholonomic Vehicle with Moving Obstacles

4.3.1 Kinematic Bicycle Model and DPCBF Constraints

We model the robot as a kinematic bicycle with state

$$x = [p_x, p_y, \theta, v]^\top,$$

where (p_x, p_y) is the position of the vehicle center of mass, θ is the heading angle, and v is the forward speed. The control inputs are longitudinal acceleration a and slip angle β , with $u = [a, \beta]^\top$. Using the small-slip approximation $\sin \beta \approx \beta$, the kinematic bicycle model can be written in control-affine form as

$$\begin{bmatrix} \dot{p}_x \\ \dot{p}_y \\ \dot{\theta} \\ \dot{v} \end{bmatrix} = \underbrace{\begin{bmatrix} v \cos \theta \\ v \sin \theta \\ 0 \\ 0 \end{bmatrix}}_{f(x)} + \underbrace{\begin{bmatrix} 0 & -v \sin \theta \\ 0 & v \cos \theta \\ 0 & \frac{v}{\ell_r} \\ 1 & 0 \end{bmatrix}}_{g(x)} \begin{bmatrix} a \\ \beta \end{bmatrix}. \quad (4.4)$$

For each moving obstacle, we instantiate the CBF constraint using the Dynamic Parabolic Control Barrier Function (DPCBF) of [17]. DPCBF provides a strong barrier-function candidate for the kinematic bicycle model with moving obstacles by formulating the unsafe set in terms of the relative position, relative heading, and relative velocity between the robot and obstacle. Although this CBF formulation is well designed, it can still get stuck in local minima and produce undesired behavior, especially when dealing with multiple conflicting constraints. In this work, DPCBF provides the barrier function candidates $h_i(x)$, while our learning framework adapts the parameters of the CBF-QP that uses these constraints.

4.3.2 Expert and Online Planners

To generate supervision, we use an expensive expert planner and a cheaper online planner. The expert is a state-time lattice planner that constructs a graph of dynamically feasible motion primitives, computed using numerical optimal control, and searches this graph with the A* algorithm. As in search-based kinodynamic motion planning, motion primitives make

$$\begin{aligned}
u^* = \arg \min_{u \in \mathbb{R}^n} \quad & \frac{1}{2}(u - u_{\text{ref}})^\top Q_\theta(z)(u - u_{\text{ref}}) + q_\theta(z)^\top u \\
\text{s.t.} \quad & L_f h_1(x) + L_g h_1(x)u \geq -\alpha_{\theta,1}(h_1(x); z) \\
& \vdots \\
& L_f h_k(x) + L_g h_k(x)u \geq -\alpha_{\theta,k}(h_k(x); z) \\
& u_{\min} \leq u \leq u_{\max}.
\end{aligned} \tag{4.5}$$

The learned policy is defined by the optimizer of this problem:

$$\pi_{\text{safe},\theta}(z, u_{\text{ref}}) = u^*.$$

Here, $Q_\theta(z, u_{\text{ref}})$ defines the relative weighting of control deviations in the quadratic objective, $q_\theta(z, u_{\text{ref}})$ is a learned residual affine term that can steer the unconstrained optimizer, and $\alpha_{\theta,i}(\cdot; z, u_{\text{ref}})$ is the learned class- $\mathcal{K}$ function used to shape the CBF constraint for obstacle i . The neural network does not directly output the executed control. Instead, it outputs the QP parameters

$$Q_\theta(z, u_{\text{ref}}), \quad q_\theta(z, u_{\text{ref}}), \quad \{\alpha_{\theta,i}(\cdot; z, u_{\text{ref}})\}_{i=1}^k,$$

which determine how the safety filter modifies u_{ref} . This gives the learned policy a structured form: the controller remains constrained by the model-based CBF-QP, while learning how the QP should adapt to the local environment and task context.

4.3.4 Network Architecture

We use an OptNet-style architecture [1], where a differentiable quadratic program forms the final layer of the policy network. Gradients are backpropagated through the QP solution, allowing the CBF-QP parameters to be trained end-to-end with standard deep learning tools. The network receives a local observation z consisting of the robot state, goal information, nominal control u_{ref} , and features for the K nearest obstacles. Each obstacle is represented in the robot frame using relative position, relative velocity, and radius, with padding used when fewer than K obstacles are observed.

Since the nearby obstacles form an unordered set, we encode them with an attention-style obstacle encoder. The encoder produces both a pooled obstacle context, summarizing the local scene, and per-obstacle embeddings, preserving information needed to parameterize individual CBF constraints. The pooled obstacle context is concatenated with the robot state, goal information, and u_{ref} , and passed through an MLP that outputs the positive diagonal entries of Q_θ and the residual affine term q_θ . The positive diagonal parameterization ensures that the quadratic objective remains positive semidefinite. In parallel, a separate

MLP takes each per-obstacle embedding together with the global context and outputs a corresponding value $\alpha_{\theta,i}$ for each nearby obstacle constraint. The resulting QP is then solved to produce the executed control.

4.3.5 Imitation Learning

We train the learner using an interactive imitation learning procedure inspired by DAgger [21] and AggreVaTe [20]. During rollouts, the learner visits states induced by its current policy, while a privileged expert planner is queried to provide both an expert action u_E and an estimate of the expert cost-to-go J_E . The expert actions provide direct behavior-cloning supervision, while the cost-to-go labels are used to train a critic $Q_\psi(z, u)$ by supervised regression. The critic then provides a deterministic policy-gradient signal that encourages the learned CBF-QP policy to choose actions with low predicted expert cost-to-go.

Given an aggregated dataset

$$\mathcal{D} = \{(z_j, u_{\text{ref},j}, u_{E,j}, J_{E,j})\}_{j=1}^{N_D},$$

we train the critic with

$$\mathcal{L}_Q(\psi) = \frac{1}{|\mathcal{D}|} \sum_j (Q_\psi(z_j, u_{E,j}) - J_{E,j})^2. \quad (4.6)$$

The learner is trained with a hybrid behavior cloning and critic-guided objective. For compactness, let

$$u_{\theta,j} \triangleq \pi_{\text{safe},\theta}(z_j, u_{\text{ref},j}).$$

We optimize

$$\mathcal{L}_\pi(\theta) = \frac{1}{|\mathcal{D}|} \sum_j [\|u_{\theta,j} - u_{E,j}\|_2^2 + \lambda Q_\psi(z_j, u_{\theta,j})], \quad (4.7)$$

where $\lambda \geq 0$ controls the contribution of the critic term. Early in training, we place more weight on the behavior-cloning term because the critic is still being fit from limited data and the expert action labels provide a stable supervised signal. As training progresses and the critic becomes more reliable, we increase λ , giving more weight to the cost-to-go objective. This annealing is useful because behavior cloning only penalizes instantaneous action mismatch and may not distinguish between errors with very different long-horizon consequences. In particular, in states near inevitable collision or other low-recoverability regions, a small action error can lead to large downstream cost. The critic term provides a more expressive training signal in these cases by biasing the policy away from actions that lead to high expert cost-to-go.

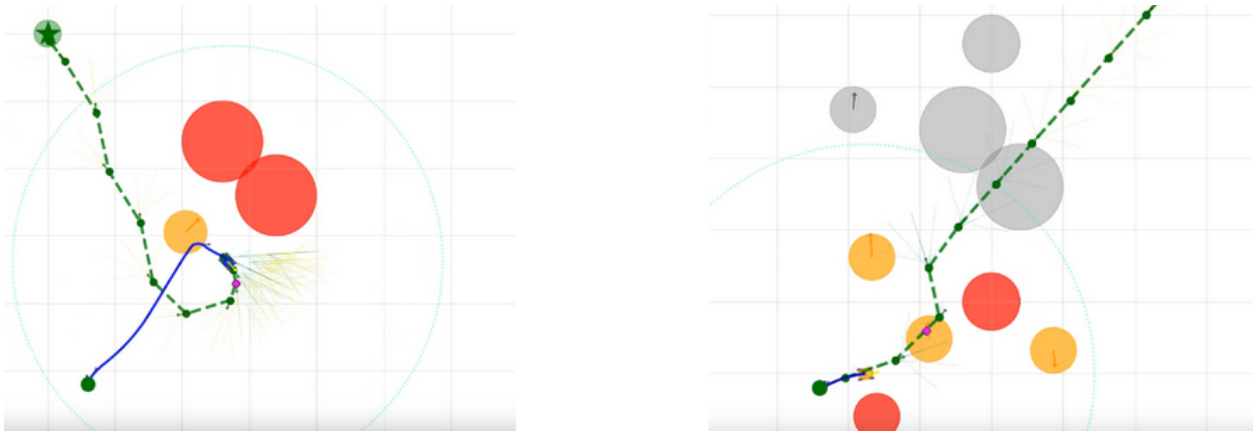


Figure 4.2. Challenging obstacle configurations where untrained CBF-QP gets trapped in local minima, even with planner guidance.

4.3.6 Results

We evaluate the proposed method in randomly generated dynamic navigation environments. During training, we sample 20 environments from a distribution over initial obstacle configurations, robot start states, and goal positions. Each environment contains between 5 and 10 moving obstacles, with obstacle positions and velocities sampled independently from the environment distribution. The performance objective is time-to-goal, measured as the number of seconds required for the robot to reach the goal without collision. Failure is defined as collision or reaching a maximum time threshold. After training, we evaluate on 5 held-out environments sampled from the same distribution. We compare three controllers: the privileged expert planner, the online planner filtered by an untrained CBF-QP, and the online planner filtered by the trained CBF-QP. This comparison isolates the effect of learning the CBF-QP parameters, since both learner variants use the same nominal online planner and differ only in whether the safety filter parameters are learned from expert supervision.

Table 4.1. Average planning statistics for the expert and online planners per query.

	Expert	Online
# Nodes expanded	1148	53
# Collision checks	41480	1988

Across the held-out evaluation environments, the navigation stack with the learned CBF-QP substantially improves performance over the same online planner with an untrained CBF-QP. As shown in Fig. [Figure 4.4](#), the trained CBF-QP achieves a relative trajectory cost of 1.26 with respect to the expert, compared to 1.89 for the untrained CBF-QP, corresponding

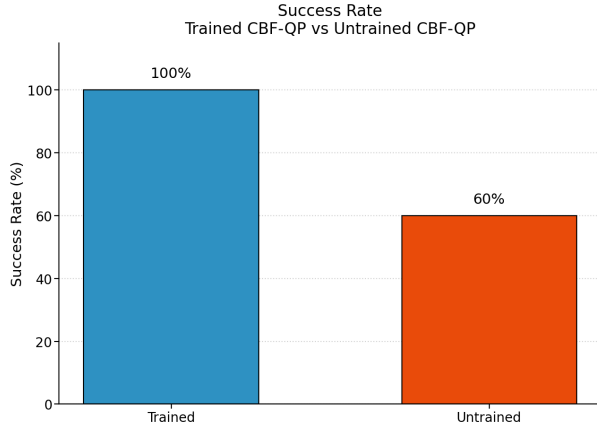


Figure 4.3. Average Success Rate between trained and untrained methods.

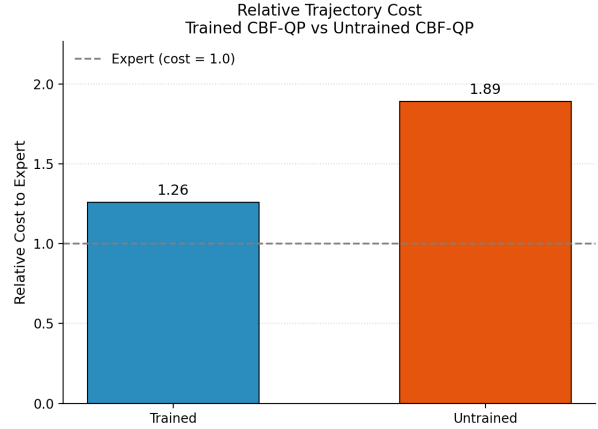


Figure 4.4. Average Performance Comparison (Successful Runs).

to a 33.3% reduction in cost. The trained model also improves the success rate from 60% to 100%, as shown in Fig. Figure 4.3. These gains are achieved even though the learner is guided by a substantially less informed online planner. Table Table 4.1 shows that the online planner expands roughly 95.4% fewer nodes and performs roughly 95.2% fewer collision checks than the expert planner on average. This suggests that learning state-dependent CBF-QP parameters enables the low-level safety filter to recover much of the expert’s behavior while retaining the computational advantages of the cheaper planning representation.

4.4 Case Study 2: Constrained, Reactive, Manipulator Planning

4.4.1 Kinematic Model and CBF Formulation

Collision-free motion planning for robot manipulators is a challenging problem due to the high-dimension of the configuration space (often 6-7 DOF), requiring efficient sampling based methods [cite RRT, RRT-connect], informative heuristic search [wA*], or optimization (chomp, stomp) to make the computation tractable. Additionally, planning for these high-dimensional systems in dynamic environments becomes even more challenging for classical methods, as the robot must adapt its motion immediately to maintain a safe distance from obstacles. Recently, deep learning based policies [deep reactive policy, neural MP], have shown impressive generalization and reaction time in these environments (due to planning being distilled into a single forward pass of the neural network), but are quite data inefficient and cannot satisfy hard constraints at test-time. To this end, we extend our framework to manipulation settings, to try to inherit the strengths of both the neural and

model-based planning approaches we discussed above. With a modest amount of expert supervision [curobo], we develop a hierarchical framework in which a simple differential inverse kinematics-based operational space controller guides the end-effector greedily towards target poses, and a learned CBF-QP joint space policy enforces constraint satisfaction (environment collision, singularity-avoidance, joint limits, self-collision) while reshaping the cost-landscape to guide the robot around local minima. We demonstrate the effectiveness of our policy on a challenging shelf reaching environment, and a bimanual manipulation environment where each arm is independently controlled via teleoperation.

4.4.1.1 Serial Manipulator and Collision-Sphere Kinematics

Consider an n -DOF serial manipulator with joint configuration $\mathbf{q} = [q_1, \dots, q_n]^\top \in \mathbb{R}^n$. Its forward kinematics map $\mathbf{q}$ to the end-effector pose

$$T_{ee}(\mathbf{q}) = \begin{bmatrix} R_{ee}(\mathbf{q}) & \mathbf{p}_{ee}(\mathbf{q}) \\ \mathbf{0}^\top & 1 \end{bmatrix} = T_1(q_1) \cdots T_n(q_n) T_{\text{tool}} \in SE(3), \quad (4.8)$$

where $R_{ee}(\mathbf{q}) \in SO(3)$ and $\mathbf{p}_{ee}(\mathbf{q}) \in \mathbb{R}^3$ are the end-effector orientation and position. Each $T_i(q_i)$ maps between consecutive link frames, so their composition maps the robot base frame to the tool frame. The geometric Jacobian $J_{ee}(\mathbf{q}) \in \mathbb{R}^{6 \times n}$ relates joint velocity to the end-effector twist:

$$\begin{bmatrix} \mathbf{v}_{ee} \\ \boldsymbol{\omega}_{ee} \end{bmatrix} = J_{ee}(\mathbf{q}) \dot{\mathbf{q}}. \quad (4.9)$$

We use a velocity-controlled model,

$$\dot{\mathbf{q}} = \mathbf{u}, \quad (4.10)$$

where $\mathbf{u} \in \mathbb{R}^n$ is the commanded joint velocity.

We approximate the robot geometry by spheres rigidly attached to its links. For collision sphere i , with radius r_i , its center in the base frame is

$$\mathbf{p}_i(\mathbf{q}) = \text{trans}(T_{\ell(i)}(\mathbf{q}) T_{s_i}^{\ell(i)}), \quad J_i(\mathbf{q}) = \frac{\partial \mathbf{p}_i}{\partial \mathbf{q}}, \quad (4.11)$$

where $\ell(i)$ is the supporting link, $T_{s_i}^{\ell(i)}$ is the fixed link-to-sphere transform, and $J_i(\mathbf{q}) \in \mathbb{R}^{3 \times n}$ is the translational point Jacobian. Consequently, $\dot{\mathbf{p}}_i = J_i(\mathbf{q}) \mathbf{u}$. This relation is the central step that converts nonlinear Cartesian geometry into inequalities that are affine in the joint-velocity command.

4.4.1.2 Environment and Workspace Collision Barriers

Let obstacle sphere j have center $\mathbf{o}_j \in \mathbb{R}^3$ and radius r_j , and let $d_{\text{safe}} > 0$ be the required clearance. We define the signed-clearance barrier

$$h_{ij}^{\text{env}}(\mathbf{q}) = \|\mathbf{p}_i(\mathbf{q}) - \mathbf{o}_j\|_2 - (r_i + r_j + d_{\text{safe}}). \quad (4.12)$$

Positive, zero, and negative values respectively denote safe separation, the safety boundary, and a clearance violation. Defining $\mathbf{d}_{ij} = \mathbf{p}_i - \mathbf{o}_j$ and $\hat{\mathbf{d}}_{ij} = \mathbf{d}_{ij}/\|\mathbf{d}_{ij}\|_2$, the chain rule gives

$$\dot{h}_{ij}^{\text{env}} = \hat{\mathbf{d}}_{ij}^\top J_i(\mathbf{q}) \mathbf{u}.$$

The relative-degree-one exponential CBF condition is therefore

$$\hat{\mathbf{d}}_{ij}^\top J_i(\mathbf{q}) \mathbf{u} + \alpha_{ij} h_{ij}^{\text{env}}(\mathbf{q}) \geq 0, \quad (4.13)$$

for $\alpha_{ij} > 0$. Equivalently, the corresponding QP row is $-\hat{\mathbf{d}}_{ij}^\top J_i \mathbf{u} \leq \alpha_{ij} h_{ij}^{\text{env}}$. It restricts the component of sphere velocity directed toward the obstacle while leaving tangential and separating motion comparatively unconstrained. In practice, only the nearest relevant sphere pairs need be retained; pair selection does not change the barrier derivation.

Planar workspace boundaries have the same form. For example, if $p_{i,z}$ is the vertical coordinate of sphere i and $z_{\min}$ is a ground height, then

$$h_i^{\text{ground}}(\mathbf{q}) = p_{i,z}(\mathbf{q}) - r_i - z_{\min}, \quad J_{i,z}(\mathbf{q}) \mathbf{u} + \alpha_{\text{ground}} h_i^{\text{ground}}(\mathbf{q}) \geq 0, \quad (4.14)$$

where $J_{i,z}$ is the vertical row of the point Jacobian.

4.4.1.3 Self-Collision and Joint-Limit Barriers

For two robot spheres i and j , define

$$h_{ij}^{\text{self}}(\mathbf{q}) = \|\mathbf{p}_i(\mathbf{q}) - \mathbf{p}_j(\mathbf{q})\|_2 - (r_i + r_j + d_{\text{self}}). \quad (4.15)$$

With $\hat{\mathbf{d}}_{ij}^{\text{self}} = (\mathbf{p}_i - \mathbf{p}_j)/\|\mathbf{p}_i - \mathbf{p}_j\|_2$, both centers move, and the CBF condition becomes

$$\left(\hat{\mathbf{d}}_{ij}^{\text{self}}\right)^\top (J_i(\mathbf{q}) - J_j(\mathbf{q})) \mathbf{u} + \alpha_{\text{self}} h_{ij}^{\text{self}}(\mathbf{q}) \geq 0. \quad (4.16)$$

The same relative-position construction handles inter-arm collision in the bimanual system: the relative Jacobian is $J_{\text{rel}} = J_{\text{left}} - J_{\text{right}}$, with columns placed according to the joints controlled by each arm.

For the admissible range $q_i^{\min} \leq q_i \leq q_i^{\max}$, including a margin m_q , the lower and upper configuration barriers are

$$h_i^{\text{low}} = q_i - (q_i^{\min} + m_q), \quad h_i^{\text{high}} = (q_i^{\max} - m_q) - q_i. \quad (4.17)$$

Since $\dot{q}_i = u_i$, their CBF constraints are

$$u_i + \alpha_q h_i^{\text{low}} \geq 0, \quad -u_i + \alpha_q h_i^{\text{high}} \geq 0. \quad (4.18)$$

The feasible velocity interval therefore contracts smoothly near each position limit. These barriers are distinct from the fixed actuator bounds

$$-\dot{\mathbf{q}}_{\max} \leq \mathbf{u} \leq \dot{\mathbf{q}}_{\max}.$$

4.4.1.4 Manipulability Barrier

To discourage kinematic singularities, let $J(\mathbf{q}) \in \mathbb{R}^{6 \times n}$ be the geometric Jacobian of one arm. The Yoshikawa manipulability measure $\mu(\mathbf{q}) = \sqrt{\det(JJ^\top)}$ is proportional to the volume of the end-effector velocity manipulability ellipsoid and approaches zero as J loses rank. For improved numerical conditioning, we use the damped log-manipulability

$$\ell(\mathbf{q}) = \frac{1}{2} \log \det A(\mathbf{q}), \quad A(\mathbf{q}) = J(\mathbf{q})J(\mathbf{q})^\top + \varepsilon I, \quad \varepsilon > 0. \quad (4.19)$$

The damping avoids products of small singular values, so ℓ is used as a numerically stable measure of proximity to singularity rather than as an exact test of matrix rank.

Using $\partial \log \det A / \partial q_i = \text{tr}(A^{-1} \partial A / \partial q_i)$ and the symmetry of A , each gradient component is

$$\frac{\partial \ell}{\partial q_i} = \text{tr} \left(\frac{\partial J}{\partial q_i} J^\top A^{-1} \right) = \text{tr} \left(\frac{\partial J}{\partial q_i} J_\varepsilon^+ \right), \quad J_\varepsilon^+ = J^\top (JJ^\top + \varepsilon I)^{-1}. \quad (4.20)$$

The Jacobian derivative tensor $\partial J / \partial \mathbf{q} = \{\partial J / \partial q_i\}_{i=1}^n$ exists and can be computed analytically from the serial-chain joint axes, joint origins, and end-effector position. Collecting these components gives $\nabla_{\mathbf{q}} \ell = [\partial \ell / \partial q_1, \dots, \partial \ell / \partial q_n]$.

For a minimum acceptable value $\ell_{\min}$, expressed directly in log-manipulability units, and margin m_{manip} , define

$$h_{\text{manip}}(\mathbf{q}) = \ell(\mathbf{q}) - \ell_{\min} - m_{\text{manip}}. \quad (4.21)$$

Its safe set is $\mathcal{C}_{\text{manip}} = \{\mathbf{q} : h_{\text{manip}}(\mathbf{q}) \geq 0\}$, and its CBF constraint is

$$\nabla_{\mathbf{q}} \ell(\mathbf{q}) \mathbf{u} + \alpha_{\text{manip}} h_{\text{manip}}(\mathbf{q}) \geq 0. \quad (4.22)$$

Far from the threshold this constraint is weak; near it, commands that reduce manipulability are restricted while commands that preserve or increase it remain available when compatible with the other constraints. For bimanual control, we construct one such barrier for each arm rather than using a single coupled Jacobian.

4.4.1.5 Learned Manipulator CBF-QP

The operational-space controller supplies a nominal command $\mathbf{u}_{\text{ref}}$. For example, a position command may first set $\mathbf{v}_{\text{des}} = K_p(\mathbf{p}_{\text{goal}} - \mathbf{p}_{ee}(\mathbf{q}))$ and then use a damped pseudoinverse of the translational end-effector Jacobian to map this velocity into joint space, optionally adding a null-space command. The learned safety filter then solves

$$\begin{aligned} \mathbf{u}^* = \arg \min_{\mathbf{u}} \quad & \frac{1}{2}(\mathbf{u} - \mathbf{u}_{\text{ref}})^\top Q_\theta(z)(\mathbf{u} - \mathbf{u}_{\text{ref}}) + \mathbf{q}_\theta(z)^\top \mathbf{u} \\ \text{s.t.} \quad & \nabla_{\mathbf{q}} h_k(\mathbf{q})\mathbf{u} + \alpha_{\theta,k}(z)h_k(\mathbf{q}) \geq 0, \quad k = 1, \dots, N_{\text{CBF}}, \\ & -\dot{\mathbf{q}}_{\text{max}} \leq \mathbf{u} \leq \dot{\mathbf{q}}_{\text{max}}. \end{aligned} \tag{4.23}$$

Here, h_k ranges over environment collision, self- and inter-arm collision, joint-limit, workspace-boundary, and manipulability barriers. The learned positive-definite, typically diagonal metric $Q_\theta(z)$ determines the cost of modifying each joint command, $\mathbf{q}_\theta(z)$ biases the solution around obstacles and local minima, and $\alpha_{\theta,k}(z) > 0$ controls the response rate of each barrier. Learning therefore changes the local objective and barrier response, while the constraints retain explicit model-based geometric structure. All constraints are affine in $\mathbf{u}$, so the online problem remains a convex quadratic program.

Collectively, forward kinematics map $\mathbf{q}$ to collision-point positions, distance functions map those positions to barriers, and point Jacobians map their time derivatives to affine constraints on $\mathbf{u}$. The manipulability constraint follows the analogous chain from $\mathbf{q}$, through $J(\mathbf{q})$ and $\ell(\mathbf{q})$, to $\nabla_{\mathbf{q}}\ell\mathbf{u}$. Subject to the standard CBF regularity conditions and QP feasibility, these constructions define a configuration-dependent safe control set. The nominal controller supplies goal-directed motion, while the learned CBF-QP selects a low-latency command within that set.

4.4.2 Results

4.5 Limitations

A key limitation of the proposed framework is that it relies on access to a reasonably accurate analytical dynamics model and differentiable CBF constraints. This makes the approach well suited for systems such as the kinematic bicycle model considered here, but potentially harder to apply to high-dimensional robots, contact-rich systems, or systems whose dynamics are difficult to model explicitly. In addition, although CBF-QPs provide a useful structured safety-filtering mechanism, the simultaneous inclusion of multiple barrier constraints can make the QP infeasible, especially in cluttered environments or states where the robot has already entered a low-recoverability region. As a result, the method does not provide hard

safety guarantees in all situations. Addressing these cases may require more advanced reachability analysis to reason about inevitable collision states and long-horizon recoverability. Related reachability-based reinforcement learning methods, which encode Hamilton-Jacobi variational inequalities into learned value functions, provide another promising way to constrain policies to safe behavior while optimizing a primary task. Comparing against and potentially integrating such reachability-based methods is an important direction for future work.

4.6 Conclusion

This work presents a differentiable optimization-based imitation learning framework for training state-dependent CBF-QP safety filters. Rather than learning an unconstrained end-to-end policy, the proposed method uses a CBF-QP as a structured policy class, allowing the controller to combine model-based safety constraints with expert-guided learning. By predicting QP objective weights, residual affine terms, and class- $\mathcal{K}$ values from local observations, the learned safety filter adapts its behavior to the robot state, obstacle configuration, and nominal planner command.

Preliminary results in dynamic navigation environments show that the learned CBF-QP improves both success rate and trajectory cost relative to an untrained CBF-QP, while relying on a substantially cheaper online planner than the expert used for supervision. These results suggest that learned CBF-QP filters can help bridge the gap between computationally expensive kinodynamic planning and real-time reactive control. Future work will evaluate the method in denser, more challenging obstacle fields, study robustness under sensing and dynamics uncertainty, and extend to higher dimensional systems like manipulators.

Chapter 5

Conclusion

Bibliography

- [1] Brandon Amos and J. Zico Kolter. Optnet: Differentiable optimization as a layer in neural networks. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 136–145. PMLR, 2017. URL <https://proceedings.mlr.press/v70/amos17a.html>.
- [2] Georges S. Aoude, Brandon D. Luders, Joshua M. Joseph, Nicholas Roy, and Jonathan P. How. Threat assessment design for driver assistance system at intersections. In *13th International IEEE Conference on Intelligent Transportation Systems (ITSC)*, pages 1855–1862, Funchal, Madeira Island, Portugal, 2010. IEEE.
- [3] Georges S. Aoude, Brandon D. Luders, Joshua M. Joseph, Nicholas Roy, and Jonathan P. How. Probabilistically safe motion planning to avoid dynamic obstacles with uncertain motion patterns. *Autonomous Robots*, 35(1):51–76, July 2013. doi: 10.1007/s10514-013-9334-3.
- [4] H. Bai, S. Cai, N. Ye, D. Hsu, and W. S. Lee. Intention-aware online pomdp planning for autonomous driving in a crowd. In *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pages 454–460, Seattle, WA, USA, 2015. doi: 10.1109/ICRA.2015.7139219.
- [5] Lars Berscheid and Torsten Kröger. Jerk-limited real-time trajectory generation with arbitrary target states. *arXiv preprint arXiv:2105.04830*, 2021. doi: 10.48550/arXiv.2105.04830. URL <https://arxiv.org/abs/2105.04830>.
- [6] B. Bäuml, T. Wimböck, and G. Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2592–2599, Taipei, Taiwan, 2010. doi: 10.1109/IROS.2010.5651175.
- [7] Peng Dai, Mausam, Daniel S. Weld, and Judy Goldsmith. Topological value iteration algorithms. *Journal of Artificial Intelligence Research*, 42:181–209, 2011. doi: 10.1613/jair.3390. URL <https://doi.org/10.1613/jair.3390>.
- [8] Y. Geng and J. Wang. Adaptive estimation of multiple fading factors in kalman filter for navigation applications. In *GPS Solution*, pages 273–279, 2008.

- [9] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics yolov8, 2023. URL <https://github.com/ultralytics/ultralytics>.
- [10] Seungsu Kim, Ashwini Shukla, and Aude Billard. Catching objects in flight. *IEEE Transactions on Robotics*, 30(5):1049–1065, 2014. doi: 10.1109/TRO.2014.2316022.
- [11] Taekyung Kim, Robin Inho Kee, and Dimitra Panagou. Learning to refine input constrained control barrier functions via uncertainty-aware online parameter adaptation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [12] Ziyang Lin and Liang Xu. Curvature-guided safety filters: State-dependent hessian-weighted projection with provable performance bounds. *arXiv preprint arXiv:2602.12603*, 2026.
- [13] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*, 2023. URL <https://arxiv.org/abs/2303.05499>.
- [14] Hengbo Ma, Bike Zhang, Masayoshi Tomizuka, and Koushil Sreenath. Learning differentiable safety-critical control using control barrier functions for generalization to novel environments. In *Proceedings of the European Control Conference (ECC)*, pages 1301–1308, 2022.
- [15] Daniel Morton and Marco Pavone. Safe, task-consistent manipulation with operational space control barrier functions. *arXiv preprint arXiv:2503.06736*, 2025.
- [16] Ramkumar Natarajan, Hanlan Yang, Qintong Xie, Yash Oza, Manash Pratim Das, Fahad Islam, Muhammad Suhail Saleem, Howie Choset, and Maxim Likhachev. Preprocessing-based kinodynamic motion planning framework for intercepting projectiles using a robot manipulator. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 1–8, 2024. arXiv:2401.08022.
- [17] Hun Kuk Park, Taekyung Kim, and Dimitra Panagou. Beyond collision cones: Dynamic obstacle avoidance for nonholonomic robots via dynamic parabolic control barrier functions. *arXiv preprint arXiv:2510.01402*, 2026.
- [18] Hardik Parwana and Dimitra Panagou. Recursive feasibility guided optimal parameter adaptation of differential convex optimization policies for safety-critical systems. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2022.

- [19] Marcel Reith-Braun, Florian Pfaff, Jakob Thumm, and Uwe D. Hanebeck. Approximate first-passage time distributions for gaussian motion and transportation models. In *Proceedings of the 26th International Conference on Information Fusion (FUSION)*, pages 1–8. IEEE, 2023. doi: 10.23919/FUSION52260.2023.10224189.
- [20] Stéphane Ross and J. Andrew Bagnell. Reinforcement and imitation learning via interactive no-regret learning. *arXiv preprint arXiv:1406.5979*, 2014. URL <https://arxiv.org/abs/1406.5979>.
- [21] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. No-regret reductions for imitation learning and structured prediction. *CoRR*, abs/1011.0686, 2010. URL <http://arxiv.org/abs/1011.0686>.
- [22] Daniele Caroline Silva, Adriano Frutuoso, Luiz Felipe Souza, and Ettore A. de Barros. Comparative analysis of innovation-based adaptive kalman filters applied to auvs navigation. In *2022 Latin American Robotics Symposium (LARS), 2022 Brazilian Symposium on Robotics (SBR), and 2022 Workshop on Robotics in Education (WRE)*, pages 31–36, 2022. doi: 10.1109/LARS/SBR/WRE56824.2022.9995869.
- [23] A. Sintov and A. Shapiro. Time-based rrt algorithm for rendezvous planning of two dynamic systems. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6745–6750, Hong Kong, China, 2014. doi: 10.1109/ICRA.2014.6907855.
- [24] Wei Xiao, Ramin Hasani, Xiao Li, and Daniela Rus. Barriernet: A safety-guaranteed layer for neural networks. *arXiv preprint arXiv:2111.11277*, 2021.
- [25] Xian ZhiWen, Hu XiaoPing, and Lian JunXiang. Robust innovation-based adaptive kalman filter for ins/gps land navigation. In *2013 Chinese Automation Congress*, pages 374–379, 2013. doi: 10.1109/CAC.2013.6775762.